



Intelligent Book Retrieval System using RAG

Overview

This project implements an intelligent **Retrieval-Augmented Generation (RAG)** system for book retrieval using the **Books to Scrape** dataset. The system combines web scraping, semantic search, vector databases, and Google's Gemini model to answer user queries accurately.

The project is developed in multiple levels, where each level extends the functionality of the previous one.

Features

- Web scraping book information from Books to Scrape.
 - Data preprocessing and chunk generation.
 - Metadata creation for each book.
 - Sentence embedding generation using SentenceTransformer.
 - Vector indexing using FAISS.
 - Semantic document retrieval.
 - Query rewriting using Gemini.
 - Query classification.
 - Structured filter extraction.
 - Metadata-based filtering.
 - Hybrid retrieval.
 - Gemini-powered answer generation.
 - Interactive Streamlit interface.
-

Dataset

Source:

Books to Scrape

The dataset contains information including:

- Book Title
 - Price
 - Rating
 - Description
 - Book URL
-

Technologies Used

- Python
 - BeautifulSoup
 - Requests
 - Pandas
 - NumPy
 - SentenceTransformers
 - FAISS
 - Google Gemini API
 - Streamlit
-

Project Structure

Level 0 – Data Collection

- Scrape book information.
 - Collect title, price, rating, description, and URL.
 - Store the dataset in JSON format.
-

Level 1 – Basic RAG Pipeline

The first RAG pipeline consists of:

1. Create text chunks.
2. Generate metadata.
3. Build document objects.
4. Generate embeddings.
5. Create FAISS vector index.
6. Retrieve Top-K relevant documents.
7. Generate answers using Gemini.

Pipeline:

User Query

↓

Embedding

↓

FAISS Retrieval



Gemini Answer

Level 2 – Query Intelligence

Level 2 enhances the system by processing user queries before retrieval.

Query Rewriting

The user query is rewritten using Gemini to make it:

- Cleaner
 - More specific
 - Easier to retrieve
 - Consistent in English
-

Query Classification

Each query is classified into categories such as:

- Recommendation
 - Comparison
 - Price-related
 - Date-related
 - Factual Lookup
 - General Question
-

Structured Filter Extraction

Important filters are extracted automatically, including:

- Rating
- Maximum Price

Example:

```
{  
  "rating": "Five",
```

```
"max_price": 20  
}
```

Metadata Filtering

Documents are filtered using metadata before semantic retrieval.

Filtered Retrieval

Only filtered documents are searched using FAISS.

Gemini Answer Generation

Gemini generates the final response using only the retrieved documents.

Updated Pipeline:

User Query

↓

Query Rewriting

↓

Query Classification

↓

Structured Filter Extraction

↓

Metadata Filtering

↓

Filtered Retrieval (FAISS)

↓

Gemini Answer Generation



Final Answer

Level 3 – Advanced Retrieval

Level 3 extends the project by adding:

- Hybrid Retrieval
 - Dense Retrieval
 - Improved Retrieval Quality
 - Streamlit User Interface
-

Workflow

1. User enters a query.
 2. Query is rewritten.
 3. Query is classified.
 4. Filters are extracted.
 5. Documents are filtered.
 6. FAISS retrieves the most relevant documents.
 7. Gemini generates the final answer.
-

Installation

Install the required packages:

```
pip install beautifulsoup4 requests pandas numpy sentence-transformers faiss-cpu google-generativeai streamlit
```

Running the Project

Run the notebook step by step.

To launch the Streamlit interface:

```
streamlit run app.py
```

Example Query

```
Recommend five-star books under £20.
```

Example Output

- Query Rewriting
 - Query Classification
 - Filter Extraction
 - Retrieved Documents
 - Final Gemini Answer
-

Future Improvements

- Add author and category filtering.
 - Support multilingual queries.
 - Improve ranking using reranking models.
 - Store vectors in ChromaDB.
 - Deploy the application online.
-

Conclusion

This project demonstrates a complete Retrieval-Augmented Generation (RAG) pipeline for intelligent book retrieval. Starting from data collection, the system progresses through semantic indexing, intelligent query understanding, metadata filtering, vector retrieval using FAISS, and answer generation with Gemini. The final solution provides accurate, context-aware, and efficient responses while showcasing modern NLP and Generative AI techniques.